

Python Logic Building — 100 Practice Questions

Beginner → Intermediate → Challenging | Python only | No DSA

Goal: These problems are designed to improve problem-solving, conditions, loops, arithmetic reasoning, strings, patterns, recursion, and practical programming logic. They intentionally avoid standard DSA topics such as linked lists, stacks, queues, trees, graphs, heaps, and algorithm-heavy data structures.

Suggested rule: Try each problem yourself first. Do not immediately search for the solution. Write the logic on paper, make a few test cases, then code it.

Level 1 — Basics & Conditions

1. Take two numbers and print the greater number.
2. Take three numbers and print the smallest number.
3. Take a person's age and determine whether they are a child, teenager, adult, or senior citizen.
4. Take a number and determine whether it is positive, negative, or zero.
5. Take a number and determine whether it is divisible by both 3 and 5.
6. Take a year and determine whether it is a leap year.
7. Take marks in five subjects and calculate the percentage and grade.
8. Take a temperature in Celsius and classify it as freezing, cold, normal, hot, or very hot using reasonable ranges.
9. Take three angles and determine whether they can form a valid triangle.
10. Take three sides and determine whether they can form a triangle.
11. Take three sides of a triangle and determine whether it is equilateral, isosceles, or scalene.
12. Take a character and determine whether it is a vowel, consonant, digit, or special character.
13. Build a simple calculator using two numbers and an operator (+, -, *, /, %).
14. Take a month number and print the number of days in that month, handling February separately.
15. Calculate a person's final salary after applying different deductions based on salary ranges.

Level 2 — Loops & Number Logic

16. Print all numbers from 1 to n.
17. Print all even numbers from 1 to n.
18. Print all odd numbers from 1 to n.
19. Find the sum of numbers from 1 to n without using the formula $n(n+1)/2$.
20. Find the sum of all even numbers from 1 to n.
21. Find the sum of all odd numbers from 1 to n.
22. Find the factorial of a number.
23. Count the number of digits in an integer.
24. Find the sum of the digits of a number.
25. Find the product of the digits of a number.
26. Reverse an integer without converting it to a string.
27. Check whether an integer is a palindrome.
28. Check whether a number is an Armstrong number.

29. Print all Armstrong numbers between two given limits.
30. Check whether a number is prime.
31. Print all prime numbers between two given limits.
32. Find the HCF/GCD of two numbers.
33. Find the LCM of two numbers.
34. Print the first n terms of the Fibonacci sequence.
35. Check whether a number belongs to the Fibonacci sequence.
36. Find all factors of a number.
37. Count how many factors a number has.
38. Determine whether a number is a perfect number.
39. Determine whether a number is a strong number.
40. Find the digital root of a positive integer.
41. Convert a decimal number to binary manually, without using bin().
42. Convert a binary number to decimal manually, without using int(x, 2).
43. Find the largest digit in a number.
44. Find the smallest digit in a number.
45. Count how many times a particular digit occurs in a number.

Level 3 — Patterns & Nested Logic

46. Print a right-angled triangle of stars for n rows.
47. Print an inverted right-angled triangle of stars.
48. Print a centered pyramid of stars.
49. Print an inverted centered pyramid of stars.
50. Print a number triangle: 1, 12, 123, 1234, ...
51. Print a repeated-number triangle: 1, 22, 333, 4444, ...
52. Print Floyd's triangle for n rows.
53. Print a multiplication table from 1 to 10.
54. Print multiplication tables from 1 to n.
55. Print a diamond pattern using stars.
56. Print a hollow square of stars.
57. Print a hollow rectangle using stars.
58. Print a checkerboard pattern using two characters.

59. Print a pattern where each row contains decreasing numbers.

60. Print a pattern that forms a numeric palindrome.

Level 4 — Strings

61. Find the length of a string without using `len()`.

62. Count vowels, consonants, digits, and spaces in a string.

63. Reverse a string without using slicing `[::-1]`.

64. Check whether a string is a palindrome.

65. Count the frequency of a given character without using `Counter`.

66. Find the first occurrence of a character without using `index()`.

67. Remove all spaces from a string without using `replace()`.

68. Remove all occurrences of a chosen character from a string.

69. Convert lowercase letters to uppercase without using `upper()`.

70. Convert uppercase letters to lowercase without using `lower()`.

71. Count the number of words in a sentence without using `split()`.

72. Reverse the order of words in a sentence.

73. Find the longest word in a sentence.

74. Find the shortest word in a sentence.

75. Check whether two strings are anagrams without using `Counter` or sorting.

76. Find the first non-repeating character in a string.

77. Count how many times each vowel occurs in a string.

78. Remove duplicate characters while preserving their first occurrence.

79. Check whether a string contains only digits without using `isdigit()`.

80. Compress consecutive repeated characters, for example `aaabbc` → `a3b2c1`.

Level 5 — Practical Logic Challenges

81. Create a number guessing game where the user gets limited attempts.

82. Create a simple ATM program with PIN verification and balance operations.

83. Create a menu-driven program for a small restaurant bill.

84. Calculate an electricity bill using slab-based pricing.

85. Calculate a parking fee based on the number of hours parked.

86. Create a simple login system with a maximum of three attempts.

- 87.** Create a password-strength checker based on length, digits, uppercase, lowercase, and special characters.
- 88.** Generate a bill for several purchased items and apply a discount based on the total.
- 89.** Calculate the amount to be paid after GST and discount.
- 90.** Create a simple income-tax calculator using predefined slabs.
- 91.** Simulate a traffic signal using user input and print the correct action.
- 92.** Create a simple calendar-style program that determines the next day from a given date.
- 93.** Given a date, determine whether it is valid.
- 94.** Find the number of days between two dates using only basic arithmetic and conditions.
- 95.** Create a stopwatch-like countdown using a loop and user input.
- 96.** Create a menu-driven unit converter for length, temperature, and currency using fixed conversion rates.
- 97.** Simulate an exam result system that accepts marks until the user chooses to stop.
- 98.** Create a simple quiz with multiple questions, calculate the score, and display performance.
- 99.** Generate a random OTP and repeatedly ask the user until the correct OTP is entered.
- 100.** Create a simple rock-paper-scissors game against the computer.

Level 6 — Challenging Logic (Still No DSA)

- 101.** Find the second largest distinct digit in an integer without converting it to a list.
- 102.** Find the second smallest distinct digit in an integer.
- 103.** Find the longest sequence of consecutive identical characters in a string.
- 104.** Find the longest word in a sentence without using `max()`.
- 105.** Find the number that occurs most frequently in a sequence of user-entered numbers without using `collections.Counter`.
- 106.** Check whether a number can be expressed as the sum of two prime numbers.
- 107.** Find all pairs of numbers between 1 and `n` whose sum equals a given target.
- 108.** Find all three-digit numbers whose digits satisfy a given mathematical condition.
- 109.** Find the number of ways to reach a target using steps of 1 or 2, using only a direct recursive solution.
- 110.** Generate all possible three-digit numbers from three given distinct digits, without using `itertools`.
- 111.** Generate all permutations of a short string using recursion, without importing `itertools`.
- 112.** Evaluate a simple arithmetic expression containing positive integers and `+`, `-`, and `*` without using `eval()`.
- 113.** Convert a positive integer into words for values from 0 to 9999.

- 114.** Find the angle between the hour and minute hands of a clock for a given time.
- 115.** Given a number of heads and legs, determine the number of chickens and rabbits if a valid solution exists.
- 116.** Find the closest point to a given point from a small set of user-entered coordinates using Euclidean distance.
- 117.** Simulate compound population growth for n years and report the population after every year.
- 118.** Find the smallest number that is divisible by all integers from 1 to n .
- 119.** Find the sum of a mathematical series such as $1/1! + 2/2! + \dots + n/n!$.
- 120.** Create a simple text-based banking transaction simulator that processes deposits, withdrawals, and balance checks until exit.

Recommended progression

1–15: Conditions & decision making
16–45: Loops and numerical reasoning
46–60: Patterns and nested loops
61–80: String logic
81–100: Practical and challenging logic

Important: These questions are intentionally not a replacement for DSA. They are a logic-building phase you can complete before starting DSA.